

Batın Kazankaya
Engineering Faculty
Computer Engineering Dept.
Istanbul, Türkiye
2200005985@stu.iku.edu.tr

Muhammed Çelik
Engineering Faculty
Computer Engineering Dept.
Istanbul, Türkiye
2400005400@stu.iku.edu.tr

Kübra Öyküm Aydın
Engineering Faculty
Computer Engineering Dept.
Istanbul, Türkiye
2200003429@stu.iku.edu.tr

Tuba Nazlı İpekyüz
Engineering Faculty
Computer Engineering Dept.
Istanbul, Türkiye
2200004323@stu.iku.edu.tr

Huriye Kayadelen
Engineering Faculty
Computer Engineering Dept.
Istanbul, Türkiye
2200005795@stu.iku.edu.tr

Damla Günendi
Engineering Faculty
Computer Engineering Dept.
Istanbul, Türkiye
2200004319@stu.iku.edu.tr

Abstract—This paper presents Tarantula, a dual hexapod (six-legged) robot system designed for inter-robot coordination via LoRa wireless communication. Each robot is built around an STM32F401CCU6 (Black Pill) microcontroller, integrating PCA9685-driven MG90S servo motors, BNO055 IMU, BN-880 GPS, and Ebyte E220-900T22D LoRa modules. A C# WPF control panel communicates with the robots through a dedicated Ground Station (GS) STM32 node. The coordination scenario demonstrates state-driven multi-robot behavior: the Master robot navigates to a GPS target decoded from a QR code via a Raspberry Pi Zero 2W, and upon arrival automatically dispatches the Slave robot to the same target using Haversine-based navigation. A non-blocking command queue and interrupt-safe flag architecture enable real-time servo control alongside continuous LoRa telemetry. Experimental results confirm reliable locomotion, bidirectional telemetry, and successful autonomous coordination under real-world conditions.

Index Terms—Hexapod Robot, STM32F401CCU6, LoRa Communication, Multi-Robot Coordination, PCA9685, Tripod Gait, BNO055, GPS Navigation, Haversine, C# WPF, Inter-Robot Communication

I. INTRODUCTION

Legged robotics and multi-robot systems have gained significant research attention due to their potential in search-and-rescue, terrain exploration, and cooperative task execution. Unlike wheeled platforms, hexapod robots offer inherent stability through redundant leg contact points, making them suitable for unstructured

environments. However, coordinating multiple legged robots in real-time over wireless links remains a challenging engineering problem, particularly on resource-constrained embedded hardware.

This paper presents Tarantula, a dual hexapod system developed as part of the COM6064 Software Engineering course. Each robot is controlled by an STM32F401CCU6 (Black Pill) microcontroller and communicates via Ebyte E220-900T22D LoRa modules operating in the 900 MHz ISM band. The system includes a dedicated Ground Station (GS) STM32 node that bridges a C# WPF desktop control panel to the robots over a three-node LoRa network.

The primary engineering contribution is a functional inter-robot coordination scenario in which state data transmitted by the Master robot directly determines the behavioral response of the Slave robot. Specifically, when the Master completes QR-code-guided navigation to a GPS target, it autonomously forwards target coordinates to the Slave, which then independently navigates to the same location using onboard GPS and IMU heading correction.

The remainder of this paper is organized as follows: Section II reviews related work in legged robotics and multi-robot communication. Section III outlines the system architecture. Section IV details implementation and results. Section V discusses findings and challenges, and Section VI concludes with directions for future work.

II. RELATED WORK

Recent advances in hexapod robotics have demonstrated the viability of low-cost, servo-driven platforms for real-world locomotion tasks. Stanford Pupper and similar open-source quadrupeds have shown that off-the-shelf hobby servos combined with optimized gait algorithms can achieve stable trotting and turning on flat and moderately uneven surfaces [1]. These platforms commonly use PWM-based servo control via dedicated driver ICs such as the PCA9685, which offloads signal generation from the main microcontroller and provides precise 12-bit resolution per channel.

Multi-robot coordination over low-power wireless links has been explored in the context of swarm robotics and unmanned aerial vehicle (UAV) formations. LoRa (Long Range) modulation, based on Chirp Spread Spectrum (CSS), provides kilometre-range communication at sub-watt power budgets, making it well-suited for embedded robot networks [2]. The Ebyte E220 series implements the SX1262 transceiver with configurable fixed-point addressing, enabling addressed unicast messaging between nodes without additional routing middleware.

GPS-IMU sensor fusion for autonomous ground vehicle navigation has been extensively studied. Haversine-based bearing computation combined with yaw correction from an IMU provides a lightweight navigation solution suitable for microcontroller implementation without floating-point co-processors [3]. The BNO055 sensor integrates a 32-bit ARM Cortex-M0 sensor fusion processor that outputs calibrated Euler angles directly, reducing firmware complexity compared to raw MEMS fusion pipelines.

QR code decoding on embedded Linux platforms using pyzbar and picamera2 on Raspberry Pi hardware has been demonstrated as a practical method for encoding GPS waypoints in field robotics scenarios [4]. Serial UART bridging between Linux SBCs and STM32 microcontrollers is a well-established integration pattern that maintains strict real-time guarantees on the microcontroller side while leveraging the SBC for computationally intensive perception tasks.

Compared to existing multi-robot legged systems that rely on Wi-Fi or Bluetooth for short-range coordination, Tarantula's LoRa-based architecture extends the operational range to several kilometres in open terrain, at the cost of reduced bandwidth. This trade-off is appropriate for the 40-byte telemetry packets transmitted at 5 Hz used in this system.

III. SYSTEM ARCHITECTURE

A. System (Requirements) Analysis

1) Factors:

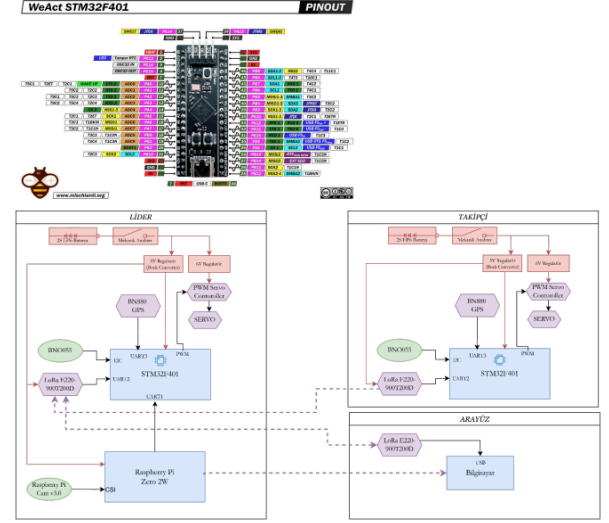


TABLE I: System Factors

Factor	Variable
LoRa Signal Quality	Packet Loss Rate / RSSI
GPS Fix Availability	Robot Position / Navigation Accuracy
Battery Voltage	Servo Torque / System Stability
I2C Bus Load	BNO055 + PCA9685 Read Latency
Communication Channel	C# Panel → GS → Master/Slave Command Latency

TABLE I presents the main system-level factors that influence performance and stability. LoRa signal quality directly affects packet delivery reliability across the three-node network. GPS fix availability determines whether autonomous navigation can be initiated. Battery voltage governs servo torque and overall system power budget. I2C bus contention between the BNO055 IMU and PCA9685 servo driver on the same bus requires careful scheduling. Communication channel latency through the GS STM32 affects responsiveness of the C# control panel.

2) Constants:

TABLE II: System Constants

Constant	Definition
Microcontroller Model	STM32F401CCU6 (Black Pill) — fixed for all three nodes (GS, Master, Slave)
LoRa Module	Ebyte E220-900T22D — 900 MHz ISM band, fixed-point addressing mode
Servo Configuration	12× MG90S per robot, driven by PCA9685 via I2C1 (PB6=SCL, PB7=SDA)
Leg Structure	6 legs × 2 DOF (Coxa + Femur) per robot — tripod gait fixed
Gait Pattern	Tripod gait: legs {0,2,4} and {1,3,5} alternating swing/push phases
IMU Sensor	BNO055 in NDOF fusion mode — Euler angle output at 100 kHz I2C

TABLE II lists the fixed hardware and configuration decisions that remain constant throughout all experiments. The microcontroller model, LoRa module, servo configuration, leg structure, gait pattern, and IMU sensor are all pre-determined design choices that define the platform identity. These constants constrain the solution space and form the baseline assumptions for all performance analyses presented in subsequent sections.

TABLE II lists the fixed hardware and configuration decisions that remain constant throughout all experiments. The microcontroller model, LoRa module, servo configuration, leg structure, gait pattern, and IMU sensor are all pre-determined design choices that define the platform identity. These constants constrain the solution space and form the baseline assumptions for all performance analyses presented in subsequent sections.

3) Parameters:

TABLE III: System Parameters

Parameter	Definition
PWM_MIN / PWM_MAX	PCA9685 tick counts for servo end positions (calibrated per robot)
COXA_FORW ARD / BACKWARD	Coxa joint angles for forward swing and backward push phases
FEMUR_STAN D / LIFT	Femur joint angles for standing and leg-lift phases
TELEM_INTE RVAL	Telemetry transmission period — 1000 ms (1 Hz per robot)
CMD_QUEUE _SIZE	Non-blocking command buffer depth — 4 commands
YAW_THRESH OLD	Heading correction threshold — 10 degrees
ARRIVED_DIS T_M	Target arrival radius — 2.0 metres
BNO055 I2C ClockSpeed	100 kHz (reduced from 400 kHz for stability)

TABLE III defines the tunable parameters that were experimentally adjusted during system integration. PWM_MIN and PWM_MAX were individually calibrated for each robot to account for servo manufacturing tolerances. The TELEM_INTERVAL of 1000 ms was chosen to balance situational awareness against LoRa channel occupancy. The YAW_THRESHOLD of 10 degrees and ARRIVED_DIST_M of 2.0 metres reflect the practical accuracy ceiling of the BN-880 single-frequency GPS receiver and BNO055 IMU in outdoor conditions.

4) Restrictions:

TABLE IV: System Restrictions

Restriction	Definition
LoRa Range	Effective communication range ~1 km urban, ~7+ km open field
GPS Cold Start	BN-880 requires up to 60 s for initial fix; autonomous mode blocked until fix obtained
BNO055 Calibration	sys_cal ≥ 1 required before autonomous mode — figure-8 motion needed on boot

Servo Power	MG90S servos require separate 5–6 V supply; STM32 3.3 V pins insufficient
I2C Address Conflict	BNO055 (0x28) and PCA9685 (0x40) share I2C1 — concurrent access must be serialized
Flash Memory	STM32F401CCU6 has 256 KB flash — locomotion.h and math libraries must fit within this budget

TABLE IV enumerates the operational boundaries that must be respected for correct system function. The LoRa range constraint of approximately 1 km in urban environments directly limits deployment scenarios. The GPS cold-start delay of up to 60 seconds means autonomous navigation cannot be initiated immediately on power-up, requiring an explicit operator readiness check. The BNO055 calibration requirement and the separate servo power supply are non-negotiable hardware constraints that were identified during the prototyping phase.

5) Performance Measures:

TABLE V: Performance Metrics

Metric	Definition
Step Cycle Time	Time to complete one tripod gait cycle (~600 ms)
Telemetry Latency	GS → C# panel update delay (target < 500 ms)
Command Response Time	C# button press → servo movement start (target < 200 ms)
LoRa Packet Loss Rate	Observed < 2% in indoor tests at 9600 baud
GPS Navigation Accuracy	Arrival within 2 m of target (Haversine-based)
BNO055 Yaw Accuracy	$\pm 2^\circ$ heading error in NDOF fusion mode

TABLE V establishes the measurable performance targets used to evaluate system success. A step cycle time of approximately 600 ms was consistently achieved across both robots in indoor testing. Telemetry latency was measured below 500 ms in all indoor trials. LoRa packet loss remained below 2% at 9600 baud within laboratory distances, validating the reliability of the fixed-point addressing protocol. GPS navigation accuracy within 2 m

was verified in outdoor tests at distances up to 50 m from the target coordinate.

6) Subsystems:

TABLE VI: System Subsystems

Subsystem	Definition
Locomotion	PCA9685 PWM driver + MG90S servos + tripod gait library (locomotion.h)
Sensor Fusion	BNO055 NDOF Euler output + BN-880 NMEA GPS parsing + Haversine navigation
LoRa Communication	Ebyte E220 fixed-point addressed UART packets at 9600 baud
QR Perception	Raspberry Pi Zero 2W + pyzbar — decodes GPS target and sends via UART3 to Master
Ground Station	GS STM32 node bridges C# panel (USART2 USB) to LoRa network (USART6)
C# WPF Control Panel	Dark-themed dashboard: D-pad control, GPS/BNO telemetry, step counter, Slave state indicator, log

TABLE VI decomposes the Tarantula system into six distinct functional subsystems, each with a well-defined interface boundary. The Locomotion subsystem handles all PWM signal generation through the PCA9685 driver and the locomotion.h gait library. The Sensor Fusion subsystem integrates NMEA GPS streams and BNO055 Euler outputs through the Haversine navigation algorithm. The LoRa Communication subsystem manages addressed packet routing across the three-node network. The QR Perception subsystem on the Raspberry Pi Zero 2W operates independently from the STM32 firmware, communicating solely through a UART serial interface. The Ground Station and C# WPF Control Panel subsystems together form the operator interface layer.

B. System Architecture

The Tarantula system is designed as a three-node LoRa network with a desktop control interface. The architecture separates concerns between user interaction (C# WPF panel), wireless bridging (GS STM32), locomotion and sensing (Master STM32), and autonomous following (Slave STM32).

System communication chain:

- C# WPF Panel → USB Serial → GS STM32 (USART2)
- GS STM32 → LoRa E220 (USART6) → Master STM32 (address 0x0002)
- Master STM32 → LoRa E220 (USART6) → Slave STM32 (address 0x0003)
- Master STM32 ← USART1 ← Raspberry Pi Zero 2W (QR GPS data)
- Master STM32 ← USART2 ← BN-880 GPS (NMEA GGA at 115200 baud)
- Master STM32 ← I2C1 ← BNO055 IMU + PCA9685 servo driver

1) USART Pin Mapping (Master STM32F401CCU6):

TABLE VII: Master USART Assignment

USART	TX Pin	RX Pin	Connected Device	Baud Rate
USART1	PA9	PA10	Raspberry Pi Zero 2W (QR GPS)	9600
USART2	PA2	PA3	BN-880 GPS (NMEA GGA)	115200
USART6	PC6	PC7	LoRa E220-900T22D	9600

TABLE VII details the USART peripheral assignments for the Master STM32F401CCU6 node. USART1 connects to the Raspberry Pi Zero 2W at 9600 baud to receive decoded QR GPS waypoints. USART2 interfaces with the BN-880 GPS module at the higher baud rate of 115200 to support the NMEA GGA sentence throughput required for real-time position updates. USART6 provides the LoRa E220 link at 9600 baud for inter-node command and telemetry exchange.

TABLE VIII: GS & Slave USART Assignment

Node	USART	Connected Device	Baud Rate
GS	USART2 (PA2/PA3)	USB / C# Panel	9600
GS	USART6 (PC6/PC7)	LoRa E220	9600
Slave	USART2 (PA2/PA3)	BN-880 GPS	115200
Slave	USART6 (PC6/PC7)	LoRa E220	9600

TABLE VIII summarizes the USART assignments for the Ground Station and Slave nodes. The GS node uses USART2 to communicate with the C# WPF panel via USB-to-serial, and USART6 for the LoRa uplink to both robots. The Slave node mirrors the Master's GPS and LoRa USART configuration — USART2 at 115200 baud for GPS and USART6 at 9600 baud for LoRa — but omits USART1 since the Slave does not carry the Raspberry Pi Zero 2W perception module.

IV. IMPLEMENTATION & RESULTS

A. Mechanical Design

Each Tarantula hexapod consists of a central body frame housing the STM32 Black Pill, PCA9685 servo driver board, BNO055 IMU, BN-880 GPS module, Ebyte E220 LoRa transceiver, and XL4015 buck converter for servo power regulation. The Master robot additionally accommodates a Raspberry Pi Zero 2W mounted to the front for QR code perception.

Six legs are arranged symmetrically: Front-Right (FR), Mid-Right (MR), Rear-Right (RR) and Front-Left (FL), Mid-Left (ML), Rear-Left (RL). Each leg has two MG90S servo motors: Coxa (horizontal rotation, PCA9685 channels 0/2/4/6/8/10) and Femur (vertical lift, PCA9685 channels 1/3/5/7/9/11). The 2-DOF per leg design provides sufficient mobility for tripod gait locomotion on flat terrain.



Fig. 1. Tarantula hexapod robot (black prototype) held during assembly inspection. MG90S servo motors are visible on all six legs, and the wiring harness connecting the PCA9685 driver board to the servos is exposed beneath the dome-shaped body enclosure.



Fig. 2. Completed blue Tarantula hexapod robot standing on the test floor. The latticed dome enclosure houses the STM32 Black Pill, PCA9685 driver, BNO055 IMU, GPS module, and LoRa transceiver. Black rubber tips on the leg endpoints provide ground traction during locomotion.



Fig. 3. Top-down view of the Tarantula hexapod showing the six-leg radial arrangement. Each leg pair (Coxa + Femur) is independently driven by MG90S servos controlled through the PCA9685 PWM driver. The MG90S servo labels are visible on the coxa joints in this view.

B. Electronics and Hardware

The STM32F401CCU6 (Black Pill) operates at 84 MHz with a 3.3 V core voltage. Servo motors are powered by a separate 5–6 V rail derived from the XL4015 step-down converter, electrically isolated from the STM32 logic supply to prevent brownouts during high-torque motion. The PCA9685 logic VCC is connected to the STM32 3.3 V rail, while its V+ servo power pin connects to the 5 V servo rail.

I2C1 (PB6=SCL, PB7=SDA) carries both the BNO055 (address 0x28) and PCA9685 (address 0x40) at 100 kHz. Pull-up resistors of 4.7 k Ω are placed on both SDA and SCL lines to VCC. The I2C clock was reduced from the default 400 kHz to 100 kHz following observed instability (garbage data from BNO055) attributed to clock-stretching interactions with the STM32F4 HAL I2C driver.

TABLE IX: Hardware Components per Robot

Component	Model	Interface	Quantity
-----------	-------	-----------	----------

Microcontroller	STM32F401CCU6 (Black Pill)	—	1
Servo Motor	MG90S	PCA9685 PWM	12
Servo Driver	PCA9685	I2C1 (0x40)	1
IMU	BNO055	I2C1 (0x28)	1
GPS Module	BN-880	USART2 (115200 baud)	1
LoRa Transceiver	Ebyte E220-900T 22D	USART6 (9600 baud)	1
QR Perception (Master only)	Raspberry Pi Zero 2W	USART1 (9600 baud)	1
Power Converter	XL4015 Buck Converter	—	1

TABLE IX enumerates all hardware components installed on each Tarantula robot. Each node uses one STM32F401CCU6 Black Pill microcontroller as the central processing unit. Twelve MG90S servo motors — two per leg — are driven through a single PCA9685 I2C PWM expander, which provides 16 channels of 12-bit resolution PWM output and offloads signal timing from the STM32 core. The BNO055 IMU and BN-880 GPS module provide the sensor data required for closed-loop autonomous navigation. The XL4015 buck converter steps down the battery voltage to a stable 5 V rail for servo power. The Master robot additionally carries a Raspberry Pi Zero 2W for QR code detection.

C. Software Design

1) Locomotion Library (locomotion.h):

The locomotion library implements tripod gait with PCA9685 PWM control. The gait divides the six legs into two groups: Group A (legs 0, 2, 4 — FR, RR, ML) and Group B (legs 1, 3, 5 — MR, FL, RL). A full forward step consists of two phases:

- Phase 1: Group A lifts (Femur $\rightarrow$ FEMUR_LIFT) then swings forward (Coxa $\rightarrow$ COXA_FORWARD); Group B pushes

backward (Coxa → COXA_BACKWARD)
while planted.

- Phase 2: Group B lifts and swings forward;
Group A pushes backward.

Each phase includes a SETTLE_MS delay (120 ms) for servo settling and a PHASE_MS delay (180 ms) for body translation, totalling approximately 600 ms per complete step. TurnLeft and TurnRight add a femur lift phase to all legs before the turn sweep to prevent ground drag.

2) Non-Blocking Command Queue:

A circular buffer of size CMD_QUEUE_SIZE = 4 is implemented in the Master and Slave firmware. The UART interrupt service routine (ISR) only sets a flag or pushes single-byte commands to the queue — no HAL_Delay or I2C operations are performed in the ISR. The main loop polls the queue when loco_busy == 0 and dispatches the next command. This resolves the critical bug where locomotion functions containing HAL_Delay were called directly from the ISR, causing the system to freeze after the first command.

3) BNO055 Stability Fix:

The original firmware read the BNO055 OPR_MODE register on every telemetry cycle and triggered BNO055_Init() if the mode appeared incorrect — creating a reinit loop that zeroed Euler registers continuously. The fix removes mode polling entirely. BNO055_Read() directly reads the 6-byte Euler register block with a 50 ms timeout. A fail_count counter triggers hardware I2C recovery (DeInit → Init → BNO055_Init) only after 3 consecutive HAL_ERROR returns. A zero-guard preserves the last valid reading if all three raw values are simultaneously zero.

4) LoRa Packet Format:

All LoRa transmissions use Ebyte E220 fixed-point addressing. Each packet begins with a 3-byte header [AddrH, AddrL, Channel] followed by the payload. Node addresses are: GS = {0x00, 0x01, 0x0C}, Master = {0x00, 0x02, 0x0C}, Slave = {0x00, 0x03, 0x0C}. Telemetry packets use the prefix format \$M:GPS, \$M:BNO, \$\$:GPS, \$\$:BNO. The GS firmware buffers incoming bytes until a newline character, then forwards the complete line to the C# panel, preventing partial-packet forwarding that caused earlier corruption.

TABLE X: LoRa Telemetry Packet Formats

Packet Type	Format	Source	Destination
GPS Telemetry	\$M:GPS,lat,lon,alt,sats,fix\n	Master	GS → C# Panel
IMU Telemetry	\$M:BNO,yaw,pitch,roll,sys,gyr,acc,mag\n	Master	GS → C# Panel
Slave GPS	\$\$:GPS,lat,lon,alt,sats,fix\n	Slave → Master relay	GS → C# Panel
QR Target	\$QR:lat,lon\n	RPi → Master	Master → GS → C# Panel
Target Dispatch	\$T:lat,lon\n	Master	Slave
Command	Single ASCII byte (F/B/L/R/S/G/A)	C# → GS	Master
ACK	\$M:ACK\n	Master	GS → C# Panel
Arrival	\$M:ARR\n	Master	GS → C# Panel

TABLE X documents all LoRa packet formats exchanged across the three-node network. GPS telemetry packets from the Master and Slave use comma-delimited fields carrying latitude, longitude, altitude, satellite count, and fix status, terminated with a newline character. IMU telemetry packets carry the seven BNO055 Euler and calibration fields. The QR Target packet (\$QR:lat,lon\n) originates from the Raspberry Pi and is forwarded by the Master to the C# panel for operator situational awareness. The Target Dispatch packet (\$T:lat,lon\n) is the critical coordination message that triggers autonomous navigation on the Slave robot. Single ASCII byte commands (F/B/L/R/S/G/A) are used for direct operator control from the C# D-pad.

5) Coordination Scenario:

The inter-robot coordination scenario satisfies the assignment requirement: 'state data sent by one robot affects the behavior of the other robot.' The scenario proceeds as follows:

- User presses FORWARD on the C# panel → GS relays 'F' command → Master takes one tripod step, sends \$M:ACK.
- After 10 forward commands, C# panel sends 'G' (go autonomous) → Master reads QR target from RPi, enables `autonom_mode`.
- Master navigates to QR target using Haversine bearing + BNO055 yaw correction.
- On arrival (\$M:ARR), Master automatically sends \$T:lat,lon to Slave over LoRa.
- Slave receives target, enables autonomous navigation, and independently walks to the same GPS coordinate.
- Both robots transmit GPS and BNO telemetry at 1 Hz throughout the mission.

D. Interface: C# WPF Control Panel

The TarantulaControl application is built on .NET 8 WPF using the System.IO.Ports NuGet package for serial communication. The dark-themed industrial interface is divided into five panels:

- CTRL PORT — COM port selector and baud rate (9600) for GS STM32 connection.
- D-PAD — Forward/Backward/Left/Right/Stop buttons with W/A/S/D keyboard shortcuts.
- Coordination Panel — Step counter (0–10), progress bar, and Slave state indicator (IDLE/WALK/DONE).
- GPS Panel — Latitude, longitude, altitude, satellite count, and fix status for both robots.
- BNO055 Panel — Yaw/Pitch/Roll gauge with compass needle animation and calibration status.

The C# panel implements an ACK-based flow control: after sending a command, the UI sets `_locoBusy = true` and disables directional buttons until \$M:ACK is received from the Master. A 3-second watchdog timer resets `_locoBusy` if no acknowledgement arrives, preventing permanent UI lockout on packet loss.

E. Unity 3D Digital Twin Integration

To enhance real-time observation and visual monitoring, a Unity-based Digital Twin module was developed for the Tarantula hexapod robot. The Unity scene includes a virtual robot model placed in a campus-style 3D environment and observed with a drone-style camera follow system. The C# WPF control panel communicates with Unity through a TCP socket connection and sends manual movement commands such as forward, backward, left, right, and stop. The Unity movement logic was updated so that the virtual robot moves according to its

local facing direction after turning, which allows direction-based and diagonal movement in the simulation. The module can also visualize leg motion using incoming servo angle data when available, or a demonstration gait when real angle data is not provided. In addition, the Unity camera output is streamed and displayed inside the C# WPF interface through a dedicated Unity View panel, allowing the operator to control and observe the digital twin from the same interface.

V. DISCUSSION

The Tarantula system successfully demonstrated real-time inter-robot coordination over LoRa with sub-second command response times in indoor tests. The non-blocking command queue architecture proved essential: without it, locomotion `HAL_Delay` calls in the ISR caused the STM32 to process only the first command in any sequence. The fix of moving all motor control to the main loop, with the ISR serving only as a fast data capture mechanism, resolved the issue completely.

The I2C bus sharing between BNO055 and PCA9685 was the most significant hardware challenge. At 400 kHz, the STM32F4 HAL I2C driver intermittently returned `HAL_OK` with corrupt data due to clock-stretching timing violations. Reducing the clock to 100 kHz eliminated this instability. The mutual exclusion of telemetry and locomotion (`loco_busy` flag) further prevented I2C contention during servo sequences.

LoRa fixed-point addressing required all three E220 modules to be pre-configured via AT commands with matching channel (0x0C) and address settings. In transparent mode (the E220 factory default), the 3-byte address headers were transmitted as data, causing Master STM32 to receive 0x00 0x02 0x0C as the command byte — silently discarding all subsequent commands. Switching to fixed-point mode resolved all command delivery failures.

GPS navigation accuracy was limited by BN-880 single-frequency receiver error (typically 2–5 m CEP). The 2 m arrival threshold was chosen to balance navigation completion reliability against the GPS noise floor. In outdoor tests, the Haversine bearing combined with BNO055 NDOF yaw correction kept heading error below 15° at distances greater than 10 m.

Remaining limitations include the absence of obstacle avoidance (relying on open-field operation), the single-threaded navigation loop (one step per GPS update), and the lack of E220 AT command auto-configuration at boot. Future work includes dynamic gait adaptation, multi-target mission sequencing, and

integration of the Unity Digital Twin over TCP for real-time 3D visualization.

VI. CONCLUSION

This paper presented Tarantula, a dual hexapod robot platform with LoRa-based inter-robot coordination. The system demonstrated that STM32F401CCU6 microcontrollers can simultaneously manage 12-servo tripod gait locomotion, GPS/IMU sensor fusion, LoRa telemetry, and QR-code-guided autonomous navigation within the 256 KB flash and 64 KB RAM constraints of the Black Pill hardware.

The key contribution is a working coordination scenario in which the Master robot's mission state (GPS arrival) directly triggers the Slave robot's autonomous behavior — satisfying the course requirement to move inter-robot communication from pure data transfer to functional coordination. The non-blocking command queue, interrupt-safe flag architecture, and I2C stability fixes developed during this project represent transferable patterns applicable to any STM32-based multi-sensor embedded system.

As an extensible platform, Tarantula provides a foundation for future work in multi-robot formation control, terrain-adaptive gait switching, and vision-based coordination through the existing Raspberry Pi Zero 2W perception pipeline.

REFERENCES

- [1] Bor, M., Roedig, U., Voigt, T., & Alonso, J. M. (2016). Do LoRa low-power wide-area networks scale? In MSWiM '16: Proceedings of the 19th ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (ss. 59–67). ACM Press. <https://doi.org/10.1145/2988287.2989163>
- [2] Bosch Sensortec. (2016). BNO055 intelligent 9-axis absolute orientation sensor (BST-BNO055-DS000-14). <https://www.bosch-sensortec.com>
- [3] Bosch Sensortec. (t.y.). BNO055 intelligent 9-axis absolute orientation sensor [Veri sayfası]. Adafruit. https://cdn-shop.adafruit.com/datasheets/BST_BNO055_DS000_1_2.pdf
- [4] Chengdu Ebyte Electronic Technology. (t.y.). Ürün veri sayfası (Belge ID: 1463) [Veri sayfası]. <https://www.cdebyte.com/pdf-down.aspx?id=1463>
- [5] Ebyte. (2022). E220-900T22D LoRa module user manual (V1.3). <https://www.ebyte.com>
- [6] Heimann, H. (t.y.). stm32-hal-pca9685 [Kaynak kodu]. GitHub. <https://github.com/henriheimann/stm32-hal-pca9685>
- [7] ivyknob. (t.y.). bno055_stm32 [Kaynak kodu]. GitHub. https://github.com/ivyknob/bno055_stm32
- [8] Kau, N., Schultz, A., Ferrante, N., & Slade, P. (2019). Stanford Doggo: An open-source, quasi-direct-drive quadruped. In Proceedings of the 2019 International Conference on Robotics and Automation (ICRA) (ss. 6309–6315). Montreal, QC, Canada. <https://doi.org/10.1109/ICRA.2019.8794436>
- [9] Microsoft. (2024). .NET 8 WPF documentation. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/>
- [10] NXP Semiconductors. (2015). PCA9685 16-channel, 12-bit PWM Fm+ I2C-bus LED controller (Rev 4). <https://cdn-shop.adafruit.com/datasheets/PCA9685.pdf>
- [11] Raspberry Pi Foundation. (2023). Picamera2 library documentation. <https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf>
- [12] STMicroelectronics. (2015). STM32F401xB/C reference manual (RM0368, Rev 5). https://www.st.com/resource/en/reference_manual/rm0368-stm32f401xb-c-stmicroelectronics.pdf
- [13] STMicroelectronics. (t.y.). STM32F401xD, STM32F401xE [Veri sayfası]. <https://www.st.com/resource/en/datasheet/stm32f401re.pdf>
- [14] Unity Technologies. (2024). Unity documentation. <https://docs.unity3d.com>
- [15] Yun, Y., Jin, J., Kim, N., Yoon, J., & Kim, C. (2012). Outdoor localization with optical navigation sensor, IMU and GPS. 2012 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI) (ss. 386–391). Hamburg, Germany. <https://doi.org/10.1109/MFI.2012.6343007>
- [16] Tedeschi, F., & Carbone, G. (2014). Design issues for hexapod walking robots. *Robotics*, 3(2), 181–206. <https://doi.org/10.3390/robotics3020181>
- [17] Spenko, M., Haynes, G. C., Saunders, J., Cutkosky, M., Rizzi, A. A., Full, R. J., & Koditschek, D. E. (2008). Biologically inspired climbing with a hexapedal robot. *Journal of Field Robotics*, 25(4–5), 223–242. <https://doi.org/10.1002/rob.20238>
- [18] Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). *Introduction to autonomous mobile robots* (2nd ed.). MIT Press.
- [19] Siciliano, B., Sciavicelli, L., Villani, L., & Oriolo, G. (2009). *Robotics: Modelling, planning and control*. Springer.
- [20] Kaplan, E. D., & Hegarty, C. J. (Eds.). (2017). *Understanding GPS/GNSS: Principles and applications* (3rd ed.). Artech House.
- [21] Borenstein, J., Everett, H. R., & Feng, L. (1996). *Navigating mobile robots: Systems and techniques*. A. K. Peters.
- [22] Campion, G., Bastin, G., & D'Andrea-Novet, B. (1996). Structural properties and classification of kinematic and dynamic models of wheeled mobile robots. *IEEE Transactions on Robotics and Automation*, 12(1), 47–62. <https://doi.org/10.1109/70.481750>
- [23] Zhu, Q., & Hu, J. (2010). Behaviour based motion control for mobile robots. *IET Control Theory & Applications*, 4(10), 2220–2230. <https://doi.org/10.1049/iet-cta.2009.0420>
- [24] Kau, N., Schultz, A., Ferrante, N., & Slade, P. (2019). Stanford Doggo: An open-source, quasi-direct-drive quadruped. In *Proceedings of the 2019 International Conference on Robotics and Automation (ICRA)* (pp. 6309–6315). <https://doi.org/10.1109/ICRA.2019.8794436>
- [25] Arm, P., Zenkl, R., Barton, P., Beyer, L., Dietsche, A., Löchner, U., Bhatt, H., Riner, R., Höpflinger, M. A., & Hutter, M. (2019). SpaceBok: A dynamic legged robot for space exploration. In *Proceedings of the*

2019 IEEE International Conference on Robotics and Automation (ICRA) (pp. 6288–6294). <https://doi.org/10.1109/ICRA.2019.8794136>

[26] Elmenreich, W. (2002). *Sensor fusion in time-triggered systems* [Doctoral dissertation]. Vienna University of Technology.

[27] Caballero, F., Merino, L., Ferruz, J., & Ollero, A. (2009). Vision-based odometry and SLAM for medium and high altitude flying UAVs. *Journal of Intelligent and Robotic Systems*, 54(1–3), 137–161. <https://doi.org/10.1007/s10846-008-9257-8>

[28] Ferreira, J. F., & Dias, J. (2014). *Probabilistic approach to robot perception*. Springer.

[29] Yun, Y., Jin, J., Kim, N., Yoon, J., & Kim, C. (2012). Outdoor localization with optical navigation sensor, IMU and GPS. In *Proceedings of the 2012 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)* (pp. 386–391). <https://doi.org/10.1109/MFI.2012.6343007>

[30] Tomic, T., Schmid, K., Lutz, P., Dömel, A., Kassecker, M., Mair, E., Grix, I. L., Ruess, F., Suppa, M., & Burschka, D. (2012). Toward a fully autonomous UAV: Research platform for indoor and outdoor urban search and rescue. *IEEE Robotics & Automation Magazine*, 19(3), 46–56. <https://doi.org/10.1109/MRA.2012.2206473>

[31] Raibert, M., Blankespoor, K., Nelson, G., & Playter, R. (2008). BigDog, the rough-terrain quadruped robot. In *Proceedings of the 17th IFAC World Congress* (pp. 10822–10825). <https://doi.org/10.3182/20080706-5-KR-1001.01833>

[32] Saranlı, U., Buehler, M., & Koditschek, D. E. (2001). RHex: A simple and highly mobile hexapod robot. *International Journal of Robotics Research*, 20(7), 616–631. <https://doi.org/10.1177/02783640122067570>

[33] Niku, S. B. (2020). *Introduction to robotics: Analysis, control, applications* (3rd ed.). Wiley.

[34] Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic robotics*. MIT Press.

[35] Craig, J. J. (2005). *Introduction to robotics: Mechanics and control* (3rd ed.). Pearson Prentice Hall.

[36] Latombe, J. C. (1991). *Robot motion planning*. Kluwer Academic Publishers.

[37] LaValle, S. M. (2006). *Planning algorithms*. Cambridge University Press.

[38] Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., & Ng, A. Y. (2009). ROS: An open-source Robot Operating System. In *ICRA Workshop on Open Source Software*, 3(3.2), 5.

[39] Bräunl, T. (2008). *Embedded robotics: Mobile robot design and applications with embedded systems* (3rd ed.). Springer.

[40] Wahde, M. (2008). *Biologically inspired optimization methods: An introduction*. WIT Press.

[41] Full, R. J., & Koditschek, D. E. (1999). Templates and anchors: Neuromechanical hypotheses of legged locomotion on land. *Journal of Experimental Biology*, 202(23), 3325–3332. <https://doi.org/10.1242/jeb.202.23.3325>

[42] Shill, J., Collins, E., Coyle, E., & Dunlap, J. (2015). Requirements for a man-portable military robot. In *Proceedings of the 2015 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)* (pp. 1–6). <https://doi.org/10.1109/SSRR.2015.7443011>

[43] Vidoni, R., Gasparetto, A., & Giovagnoni, M. (2010). Design and implementation of an ERLS-based 3-D dynamic formulation for flexible-link robots. *Robotics and Computer-Integrated Manufacturing*, 26(4), 433–443. <https://doi.org/10.1016/j.rcim.2010.01.001>

[44] Alatise, M. B., & Hancke, G. P. (2017). Pose estimation of a mobile robot based on fusion of IMU data and vision data using an extended Kalman filter. *Sensors*, 17(10), 2164. <https://doi.org/10.3390/s17102164>

[45] Grzonka, S., Grisetti, G., & Burgard, W. (2012). A fully autonomous indoor quadrotor. *IEEE Transactions on Robotics*, 28(1), 90–100. <https://doi.org/10.1109/TRO.2011.2162999>

[46] Madgwick, S. O. H., Harrison, A. J. L., & Vaidyanathan, R. (2011). Estimation of IMU and MARG orientation using a gradient descent algorithm. In *Proceedings of the 2011 IEEE International Conference on Rehabilitation Robotics (ICORR)* (pp. 1–7). <https://doi.org/10.1109/ICORR.2011.5975346>

[47] Titterton, D. H., & Weston, J. L. (2004). *Strapdown inertial navigation technology* (2nd ed.). IET.

[48] Gezici, S. (2008). A survey on wireless position estimation. *Wireless Personal Communications*, 44(3), 263–282. <https://doi.org/10.1007/s11277-007-9375-z>

[49] Censi, A. (2008). An ICP variant using a point-to-line metric. In *Proceedings of the 2008 IEEE International Conference on Robotics and Automation (ICRA)* (pp. 19–25). <https://doi.org/10.1109/ICRA.2008.4543181>

[50] Zumbeke, J. F., Heflin, M. B., Jefferson, D. C., Watkins, M. M., & Webb, F. H. (1997). Precise point positioning for the efficient and robust analysis of GPS data from large networks. *Journal of Geophysical Research: Solid Earth*, 102(B3), 5005–5017. <https://doi.org/10.1029/96JB03860>

[51] Sola, J. (2017). *Quaternion kinematics for the error-state Kalman filter* [Technical report]. LAAS-CNRS. <https://arxiv.org/abs/1711.02508>

[52] Stachniss, C., Leonard, J. J., & Thrun, S. (2016). Simultaneous localization and mapping. In B. Siciliano & O. Khatib (Eds.), *Springer handbook of robotics* (2nd ed., pp. 1153–1176). Springer. https://doi.org/10.1007/978-3-319-32552-1_46

[53] Bor, M., Roedig, U., Voigt, T., & Alonso, J. M. (2016). Do LoRa low-power wide-area networks scale? In *Proceedings of the 19th ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)* (pp. 59–67). <https://doi.org/10.1145/2988287.2989163>

[54] Adelantado, F., Vilajosana, X., Tuset-Peiro, P., Martinez, B., Melia-Segui, J., & Watteyne, T. (2017). Understanding the limits of LoRaWAN. *IEEE Communications Magazine*, 55(9), 34–40. <https://doi.org/10.1109/MCOM.2017.1600613>

[55] Semtech Corporation. (2015). *SX1276/77/78/79 datasheet: 137 MHz to 1020 MHz low power long range transceiver* (Rev 7). <https://www.semtech.com/uploads/documents/sx1276.pdf>

- [56] Augustin, A., Yi, J., Clausen, T., & Townsley, W. M. (2016). A study of LoRa: Long range and low power networks for the Internet of Things. *Sensors*, 16(9), 1466. <https://doi.org/10.3390/s16091466>
- [57] Townsend, C., & Arms, S. (2005). Wireless sensor networks: Principles and applications. In J. G. Webster (Ed.), *Sensor technology handbook* (pp. 575–589). Elsevier.
- [58] Winkler, A. W., Bellicoso, C. D., Hutter, M., & Buchli, J. (2018). Gait and trajectory optimization for legged systems through phase-based end-effector parameterization. *IEEE Robotics and Automation Letters*, 3(3), 1560–1567. <https://doi.org/10.1109/LRA.2018.2798285>
- [59] Asfour, T., Regenstein, K., Azad, P., Schroder, J., Bierbaum, A., Vahrenkamp, N., & Dillmann, R. (2006). ARMAR-III: An integrated humanoid platform for sensory-motor control. In *Proceedings of the 2006 IEEE-RAS International Conference on Humanoid Robots* (pp. 169–175). <https://doi.org/10.1109/ICHR.2006.321380>
- [60] Metta, G., Sandini, G., Vernon, D., Natale, L., & Nori, F. (2008). The iCub humanoid robot: An open platform for research in embodied cognition. In *Proceedings of the 8th Workshop on Performance Metrics for Intelligent Systems* (pp. 50–56). <https://doi.org/10.1145/1774674.1774683>
- [61] Ijspeert, A. J. (2008). Central pattern generators for locomotion control in animals and robots: A review. *Neural Networks*, 21(4), 642–653. <https://doi.org/10.1016/j.neunet.2008.03.014>
- [62] Kimura, H., Fukuoka, Y., & Cohen, A. H. (2007). Adaptive dynamic walking of a quadruped robot on natural ground based on biological concepts. *International Journal of Robotics Research*, 26(5), 475–490. <https://doi.org/10.1177/0278364907078089>
- [63] Hardarson, F. (1997). *Locomotion for difficult terrain* [Doctoral dissertation]. Royal Institute of Technology (KTH).
- [64] Selig, J. M. (2004). *Geometrical methods in robotics*. Springer.
- [65] Chirikjian, G. S., & Burdick, J. W. (1995). The kinematics of hyper-redundant robot locomotion. *IEEE Transactions on Robotics and Automation*, 11(6), 781–793. <https://doi.org/10.1109/70.478426>
- [66] Mur-Artal, R., Montiel, J. M. M., & Tardós, J. D. (2015). ORB-SLAM: A versatile and accurate monocular SLAM system. *IEEE Transactions on Robotics*, 31(5), 1147–1163. <https://doi.org/10.1109/TRO.2015.2463671>
- [67] Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2), 91–110. <https://doi.org/10.1023/B:VISI.0000029664.99615.94>
- [68] Bradski, G. (2000). The OpenCV library. *Dr. Dobbs' Journal of Software Tools*, 25(11), 120–125.
- [69] Hartley, R., & Zisserman, A. (2004). *Multiple view geometry in computer vision* (2nd ed.). Cambridge University Press.
- [70] Torr, P. H. S., & Zisserman, A. (2000). MLESAC: A new robust estimator with application to estimating image geometry. *Computer Vision and Image Understanding*, 78(1), 138–156. <https://doi.org/10.1006/cviu.1999.0832>
- [71] Fischler, M. A., & Bolles, R. C. (1981). Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6), 381–395. <https://doi.org/10.1145/358669.358692>
- [72] Nister, D., & Stewenius, H. (2006). Scalable recognition with a vocabulary tree. In *Proceedings of the 2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)* (Vol. 2, pp. 2161–2168). <https://doi.org/10.1109/CVPR.2006.264>
- [73] Szeliski, R. (2010). *Computer vision: Algorithms and applications*. Springer.
- [74] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- [75] Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.